Mock Midterm, Compilers: Full Solutions

Lexical analysis, grammars and top-down parsing (100 points)

Compiler Construction Module

Every FIRST/FOLLOW set, table and stack trace below was recomputed by a script (`check_ll1.py`) from the grammars as written.

Summary: How the paper is built

Five problems, 120 minutes. Problem 1 (30 pts) is nine short questions over the whole syllabus; Problem 2 (22) is the scanner-generator side of lexical analysis (rule order, longest match, a combined DFA, comments); Problems 3 to 5 (48) are LL(1): conditioning a grammar, FIRST/-FOLLOW, the table, the recursive-descent code and the stack machine. Each solution ends with the box that tells where the method lives in `/textbooks/compilers`. Every multiple-choice answer below includes the justification the paper requires, plus why the other options are wrong.

Problem 1: Short questions and multiple choice (30 pts)

Problem: 1.1 (3 pts): definition of a context-free grammar

Solution

(A). A CFG has productions $A \to w$ with a *single nonterminal* on the left and any string $w \in (N \cup T)^*$, possibly empty, on the right: the left side is replaced regardless of its context. (B) describes the monotone (context-sensitive) grammars of Chomsky type 1, where the left side may be a string. (C) is the *regular* (right-linear) grammars of type 3, a strict subset. (D) confuses a class with a property: context-freeness is about the shape of productions; a CFG may or may not be ambiguous ($E \to E - E \mid \mathsf{NUM}$ in 1.4 is a context-free grammar and is ambiguous).

Problem: 1.2 (4 pts): $L = \{a^n b^{2n} : n \geq 0\}$

Solution

(a) Not regular (3 pts). Suppose a DFA M with p states accepts L. Feed it a^p: among the $p+1$ prefixes $a^0, a^1, \ldots, a^p$ two reach the same state, say a^i and a^j with $i < j$. From that state the same suffixes are accepted, so $a^i b^{2i} \in L$ forces $a^j b^{2i} \in L$, but $2i \neq 2j$: contradiction. (Equivalently with the pumping lemma: take $w = a^p b^{2p}$; any split xyz with $|xy| \leq p$, $|y| \geq 1$ has $y = a^k$, and $xy^2z = a^{p+k}b^{2p} \notin L$.) Intuitively a finite automaton cannot count an unbounded number of a's to check twice as many b's.
(b) Grammar (1 pt). $S \to aSbb \mid \varepsilon$. Each use of the first rule adds one a on the left and two b's on the right; n uses then ε give $a^n b^{2n}$.

Problem: 1.3 (4 pts): derivations and parse tree for a e b d b d f c

Solution

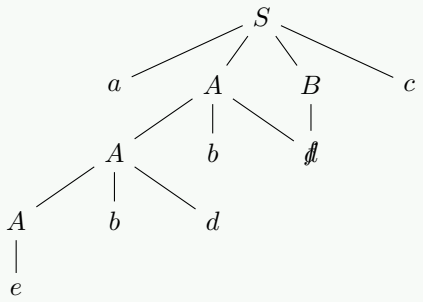
A must produce e b d b d: $A \to Abd \to Abd\,bd \to e\,bd\,bd$.
(a) Leftmost (always expand the leftmost nonterminal):

$$S \underset{lm}{\Rightarrow} aABc \underset{lm}{\Rightarrow} aAbdBc \underset{lm}{\Rightarrow} aAbdbdBc \underset{lm}{\Rightarrow} aebdbdBc \underset{lm}{\Rightarrow} aebdbdfc.$$

(b) Rightmost (always expand the rightmost nonterminal):

$$S \underset{rm}{\Rightarrow} aABc \underset{rm}{\Rightarrow} aAfc \underset{rm}{\Rightarrow} aAbdfc \underset{rm}{\Rightarrow} aAbdbdfc \underset{rm}{\Rightarrow} aebdbdfc.$$

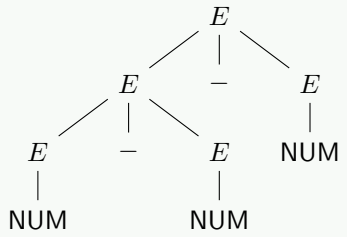
(c) Parse tree (the same tree for both derivations; only the order of expansion differs):

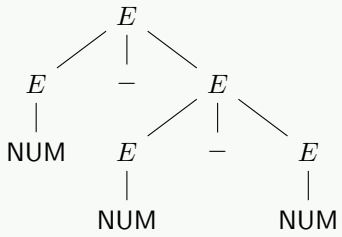


Problem: 1.4 (7 pts): the ambiguous subtraction grammar $E \to E{-}E \mid (E) \mid$ NUM

Solution

(a) Two parse trees (1 pt) for NUM - NUM - NUM:





(b) Values (2 pts). Left tree: $(10-4)-3=3$. Right tree: $10-(4-3)=9$. A compiler must pick the left tree: subtraction is **left-associative** by convention, so $10-4-3$ means $(10-4)-3$.

(c) Unambiguous, left-associative (3 pts).

$$E \to E - T \mid T, \qquad T \to (E) \mid \mathsf{NUM}.$$

Recursion on the left of $-$ makes the left operand the "rest of the chain" and the right operand a single term, so every chain nests to the left; each sentence has exactly one tree.

(d) LL(1) form (1 pt).

$$E \to TE', \qquad E' \to -TE' \mid \varepsilon, \qquad T \to (E) \mid \mathsf{NUM}.$$

A predictive parser cannot use $E \to E - T$: to decide on E it would first have to parse E, i.e. call `E()` again without consuming a token, and recurse for ever (in table terms, $\mathrm{FIRST}(E-T) = \mathrm{FIRST}(T)$: two productions in every cell of row E).

Problem: 1.5 (4 pts): $S \to Ax \mid Bx$, $A \to a \mid \varepsilon$, $B \to b \mid \varepsilon$

Solution

(a) $\mathrm{FIRST}(A) = \{a, \varepsilon\}$, $\mathrm{FIRST}(B) = \{b, \varepsilon\}$, $\mathrm{FIRST}(S) = \{a, b, x\}$ (x enters because A and B are nullable). $\mathrm{FOLLOW}(S) = \{\$\}$, $\mathrm{FOLLOW}(A) = \{x\}$, $\mathrm{FOLLOW}(B) = \{x\}$.

(b) (C), the cell $M[S, x]$. $S \to Ax$ is selected on $\mathrm{FIRST}(Ax) = \{a, x\}$ and $S \to Bx$ on $\mathrm{FIRST}(Bx) = \{b, x\}$; both contain x, so on lookahead x the parser cannot tell whether the (empty) A or the (empty) B was meant. $M[S, a]$ and $M[S, b]$ each hold one production, so (A) and (B) are wrong, and (D) is wrong because one cell is doubly filled. The language $\{x, ax, bx\}$ is LL(1) with the grammar $S \to ax \mid bx \mid x$: the defect is in the grammar, not the language.

Problem: 1.6 (4 pts): eliminate ε from $S \to (S)S \mid \varepsilon$

Solution

(a) Nullable: S only (it has an ε-production).
(b) Since $S \Rightarrow \varepsilon$, ε is in the language; keep it through a new start symbol S'. For every production, every subset of the nullable occurrences may be deleted; $S \to (S)S$ has two occurrences of S, giving four versions:

$$S' \to S \mid \varepsilon, \qquad S \to (S)S \mid (S) \mid ()S \mid ().$$

Now S has no ε-production and generates exactly the nonempty balanced-parenthesis strings; S' adds ε.

Problem: 1.7 (4 pts): NFAs versus DFAs; a regex for numeric literals

Solution

(a) (D). The subset construction turns any NFA into a DFA for the same language, and its states are *sets* of NFA states, so there can be up to 2^n of them; the blow-up is real, e.g. "the n-th symbol from the end is a" has an $(n{+}1)$-state NFA and needs 2^n DFA states. (A) is false: both recognise exactly the regular languages, and generators prefer *DFAs* because they run without backtracking. (B) has it backwards: several transitions on one symbol (and ε-moves) are what an NFA may have; a DFA has at most one. (C) is false: the number of states can go down (unreachable subsets) or up (exponentially).
(b) With $D = [0\text{-}9]$:

$$D^+ \; (\; . \; D^+ \;)? \; (\; (E \mid e) \; (+ \mid -)? \; D^+ \;)?$$

`42`, `3.14`, `6.02E23`, `1.5e-3` match. `.5` fails (the integer part D^+ is mandatory), `3.` fails (a dot must be followed by D^+), `1e` fails (an exponent must contain D^+).

Where it is in the textbooks

1.1, 1.7(a): `01_Introduction` §4 (Chomsky hierarchy) and `02-04_Lexical_Analysis_Complete` Methods 3 and 7. 1.2(a): Method 7, "distinguishable strings" (the DFA-state argument above; the pumping lemma itself is not in the book). 1.3, 1.4, 1.5: `05_Parsers` §1 (derivations, parse trees, ambiguity), §4–6 (FIRST, FOLLOW, the table), §7 (left recursion, associativity). 1.7(b): `02_Tokens` §3.5 (the method for building a token regex, with the same counter-examples). **1.6 (ε-elimination of a grammar) is not in the textbooks**: `05_Parsers` defines nullable symbols for FIRST, but never performs the transformation.

Problem 2: Lexical analysis (22 pts)

Problem: 2.1 (5 pts): tokens for `if iff<=3.14<>x12 if3 7.`

Solution

Longest match first, then rule order on ties.

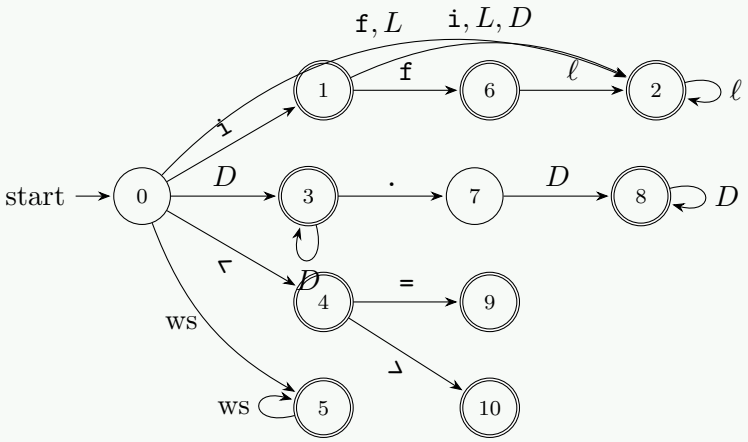
Lexeme	Candidates	Decision	Token
`if`	`if` (2), ID (2)	tie, `if` listed first	IF `if`
`iff`	`if` (2), ID (3)	longest	ID `iff`
`<=`	LT (1), LE (2)	longest	LE `<=`
`3.14`	INT (1), REAL (4)	longest	REAL `3.14`
`<>`	LT (1), NE (2)	longest	NE `<>`
`x12`	ID (3)		ID `x12`
`if3`	`if` (2), ID (3)	longest	ID `if3`
`7`	INT (1); REAL needs a digit after the dot	back up to the last accept	INT `7`
`.`	nothing can start with `.`	**lexical error**, skipped	–

So: IF, ID(iff), LE, REAL(3.14), NE, ID(x12), ID(if3), INT(7), then the error on the final `.` (the scanner had read `7.` hoping for a REAL, found the end of input, and backed up to 7).

Problem: 2.2 (7 pts): the combined DFA, keyword versus identifier, longest match

Solution

Character classes: `i`, `f`, L (other lowercase letter), D (digit), `.`, `<`, `=`, `>`, `ws`. "ℓ" below abbreviates $\{\texttt{i}, \texttt{f}, L, D\}$, the identifier-continuation class.



1: ID	2: ID	6: **IF** (also ID)	3: INT	8: REAL	4: LT
9: LE	10: NE	5: skip	7: not accepting	0: start	

The lexeme `if`. State 6 is reached by the string `if` and is accepting for

both the keyword rule and the identifier rule. The generator labels each accepting state with the *first-listed* rule among those it accepts, so state 6 returns IF; one more letter or digit moves to state 2, which is identifier-only, so `iff` and `if3` are identifiers.

Longest match. The scanner keeps following transitions as long as one exists, remembering the last accepting state it passed and the input position where it passed it. When it gets stuck (no transition, or end of input), it backs up to that remembered position, returns that state's token, and restarts the DFA in state 0 from there. On `7.` it reaches state 7 (not accepting), gets stuck, and backs up to state 3: INT 7, then `.` is rescanned from state 0 and has no transition: error.

Problem: 2.3 (6 pts): comments over $\{/, *, x\}$

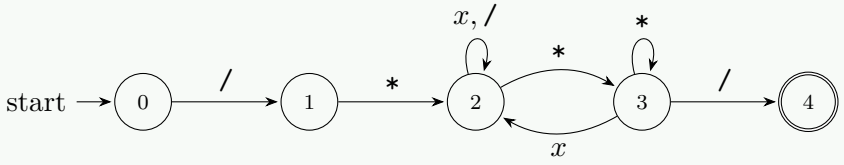
Solution

(a) Regular expression (2 pts). A comment is `/*`, then a body that never contains `*/`, then the first `*/`:

$$/\,*\,(\,x\;\mid\;/\;\mid\;*^{+}\,x\,)^{*}\,*^{+}\,/$$

Inside the body a run of stars may only be followed by x (a `/` would close the comment); the closing is a run of one or more stars followed by `/`, which also covers `**/`.

(b) DFA (3 pts).



State 2: inside the body, last character not a star. State 3: inside, just read one or more stars. State 4: the first `*/` has been read; it has no outgoing transitions, so exactly the comments are accepted (any symbol after the closing `*/` is rejected). Missing transitions (e.g. state 0 on x) go to a dead state.

(c) Nested comments (1 pt). No. Accepting `/* a /* b */ c */` as one comment requires knowing how many `/*` are still open, an unbounded count; a DFA with k states cannot distinguish depth k from some smaller depth (the strings $(/*)^1, (/*)^2, \ldots$ are pairwise distinguishable by the number of `*/` needed to close). Nested comments are the balanced-parenthesis language in disguise: context-free, so a lexer needs a counter or a stack (flex: a start condition plus a depth variable).

Problem: 2.4 (4 pts): the identifier rule moved above `if`

Solution

(A): ID for both. On `if` the two rules match the same length, the tie goes to the rule listed first, now the identifier rule. On `iff` the identifier rule is the only longest match, as before. (B) is wrong because order is exactly what decides a tie; (C) reverses the two cases; (D) is wrong because two rules matching is the normal situation, resolved by the two rules, never an error. The keyword rule has become unreachable (flex warns "rule cannot be matched"): this is why keyword rules are always listed *before* the identifier rule, or keywords are looked up in a table after an identifier is matched.

Where it is in the textbooks

2.1, 2.4: `02_Tokens` §4.3 (how conflicts are resolved: longest match, then first rule) and `02-04_Complete` §5 "Two rules and a safety net". 2.2: `02-04_Complete` §9 (many tokens in one automaton, accepting states labelled by the first rule) and §10 (the scan-and-backtrack loop), `04_Practice` §5.1 (the combined DFA). 2.3(c): `02-04_Complete` §13 (nested comments need a counter) and Method 7. **2.3(a,b), the comment regex and its DFA, are only implicit**: the book handles comments with lexer states (`02_Tokens` §4.5), not with a single regular expression; the pattern above is worth adding.

Problem 3: LL(1) parsing, nested blocks (19 pts)

Problem: 3.1 (8 pts): why not LL(1), and the fix

Solution

(a) (C). *Stmts* → *Stmts Stmt* is left-recursive. With one token of lookahead the parser sees begin or id and must choose between *Stmts* → *Stmts Stmt* (its FIRST contains FIRST(*Stmt*) = {begin, id}) and *Stmts* → ε (selected on FOLLOW(*Stmts*), which contains begin, id and end): both cells $M[Stmts, \mathsf{begin}]$ and $M[Stmts, \mathsf{id}]$ hold two productions, and a recursive-descent `Stmts()` would call itself without consuming input. (A) is wrong: the grammar is unambiguous, a statement list has exactly one (left-nested) tree. (B) is wrong: *Stmt* → *Block* starts with begin and *Stmt* → *Simple* with id; disjoint. (D) is wrong: the language is given by a CFG, so it is context-free, and (b) exhibits an LL(1) grammar for it.

(b) Replace the left recursion by right recursion; the language (sequences of statements) is unchanged and no new nonterminal is needed:

1	*Block* → begin *Stmts* end	4	*Stmt* → *Block*
2	*Stmts* → *Stmt Stmts*	5	*Stmt* → *Simple*
3	*Stmts* → ε	6	*Simple* → id ;

Problem: 3.2 (5 pts): FIRST and FOLLOW

Solution

	FIRST	FOLLOW
Block	{begin}	{$, begin, id, end}
Stmts	{begin, id, ε}	{end}
Stmt	{begin, id}	{begin, id, end}
Simple	{id}	{begin, id, end}

Reasoning: FOLLOW(*Stmts*) = {end} from rule 1 (and rule 2 puts FOLLOW(*Stmts*) into itself). FOLLOW(*Stmt*) = FIRST(*Stmts*) $\setminus \{\varepsilon\} \cup$ FOLLOW(*Stmts*) from rule 2, since *Stmts* is nullable. *Block* and *Simple* are at the end of a *Stmt* production, so they inherit FOLLOW(*Stmt*); *Block* is also the start symbol, hence $.

Problem: 3.3 (6 pts): the LL(1) table

Solution

	begin	end	id	;	$
Block	1				
Stmts	2	3	2		
Stmt	4		5		
Simple			6		

Rule 3 goes under FOLLOW(*Stmts*) = {end}. No cell holds two entries: the grammar is LL(1).

Where it is in the textbooks

All of Problem 3: `05_Parsers` §4 (nullable, FIRST), §5 (FOLLOW), §6.2–6.3 (table and the LL(1) condition), §7.1 (left recursion, including the "no new nonterminal" right-recursive rewrite for lists), §8 (the checklist "is this grammar LL(1)?").

Problem 4: Recursive descent, parse tree and stack (16 pts)

Problem: 4.1 (3 pts): the grammar in the code and its selection sets

Solution

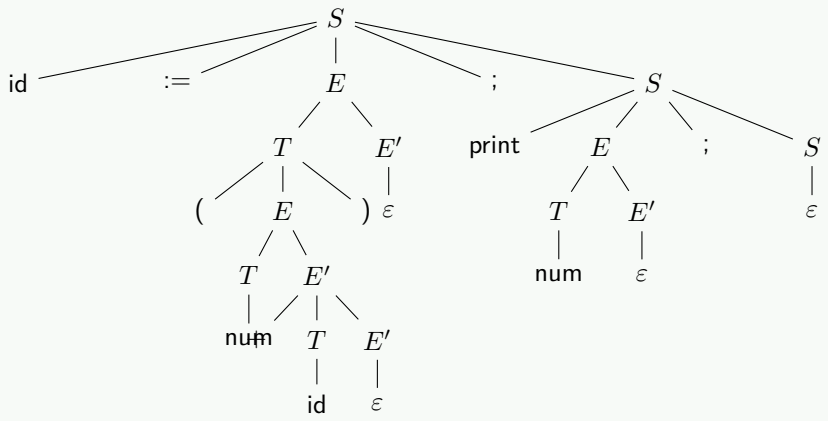
	Production	Selected by
1	$S \to \mathsf{ID} := E\ ;\ S$	{ID}
2	$S \to \mathsf{PRINT}\ E\ ;\ S$	{PRINT}
3	$S \to \varepsilon$	FOLLOW(S) = {EOF}
4	$E \to T\ E'$	FIRST(T) = {ID, NUM, (}
5	$E' \to +\ T\ E'$	{+}
6	$E' \to \varepsilon$	FOLLOW(E') = FOLLOW(E) = {;,)}
7	$T \to \mathsf{ID}$	{ID}
8	$T \to \mathsf{NUM}$	{NUM}
9	$T \to (\ E\ )$	{(}

FOLLOW(E): E is followed by ; in rules 1 and 2 and by) in rule 9; E' ends rules 4 and 5, so it inherits FOLLOW(E). Note that the code's `Eprime()` returns on *any* token other than +, which is more permissive than the selection set {;,)}; the difference shows up in 4.4.

Problem: 4.2 (4 pts): `id := ( num + id ) ; print num ;`

Solution

(a) Parse tree (3 pts).



(b) Calls for the first statement (1 pt), in order (indentation shows nesting):

```
S()
  match(ID)  match(ASSIGN)
  E()
    T()
      match(LPAREN)
      E()
        T()  match(NUM)
        Eprime()  match(PLUS)  T()  match(ID)  Eprime()  [lookahead ')' :
      returns]
      match(RPAREN)
    Eprime()  [lookahead ';' : returns]
```

```
match(SEMI)
S()   -- the second statement starts here
```

Problem: 4.3 (4 pts): stack parse of `print ( id ) + num ;`

Solution

Table = the selection sets of 4.1. Stack top at the right; $ marks the bottom and the end of input.

	Stack	Remaining input	Action
1	$ S	`print ( id ) + num ; $`	$M[S, \mathsf{print}] = 2$: pop S, push S ; E print
2	$ S ; E print	`print ( id ) + num ; $`	match print
3	$ S ; E	`( id ) + num ; $`	$M[E, (] = 4$: push $E'\,T$
4	$ S ; $E'\,T$	`( id ) + num ; $`	$M[T, (] = 9$: push $)\,E\,($
5	$ S ; E') E (	`( id ) + num ; $`	match (
6	$ S ; E') E	`id ) + num ; $`	$M[E, \mathsf{id}] = 4$
7	$ S ; E') $E'\,T$	`id ) + num ; $`	$M[T, \mathsf{id}] = 7$
8	$ S ; E') E' id	`id ) + num ; $`	match id
9	$ S ; E') E'	`) + num ; $`	$M[E',)] = 6$: pop E'
10	$ S ; E')	`) + num ; $`	match)
11	$ S ; E'	`+ num ; $`	$M[E', +] = 5$: push $E'\,T\,+$
12	$ S ; $E'\,T\,+$	`+ num ; $`	match +
13	$ S ; $E'\,T$	`num ; $`	$M[T, \mathsf{num}] = 8$
14	$ S ; E' num	`num ; $`	match num
15	$ S ; E'	`; $`	$M[E', ;] = 6$: pop E'
16	$ S ;	`; $`	match ;
17	$ S	`$`	$M[S, \$] = 3$: pop S
18	$	`$`	**accept**

Problem: 4.4 (2 pts): where the two parsers detect the errors

Solution

`id := num num ;` *Recursive parser:* `S()` matches ID and :=, calls `E()` → `T()` matches NUM; `Eprime()` sees NUM, which is not +, and *returns silently*; back in `S()`, `match(SEMI)` finds NUM: the error is raised in `match`, called from `S()` for the semicolon of the first statement. *Stack parser:* after matching `num` the stack is $\$\,S\,;E'$ with input **`num ; $`**; the cell $M[E', \mathsf{NUM}]$ is empty: error at E'. Same token, different place: the code's `Eprime()` has no `error()` branch, so it accepts ε on *any* non-+ token and leaves the check to the next `match`; the table uses the exact selection set $\mathrm{FOLLOW}(E') = \{;,)\}$ and refuses NUM immediately.

`id := + ;` *Recursive parser:* `S()` → `E()` → `T()` sees +: `error()` in `T()`, called from `E()` inside the first `S()`. *Stack parser:* stack $\$\,S\,;E$, input `+ ; $`; $M[E, +]$ is empty: error at E, one expansion earlier than the code. Reason: `E()` has a single alternative and checks nothing before calling `T()`, whereas the table has an explicit row for E whose only filled cells are ID, NUM, (.

Problem: 4.5 (3 pts): `void E() { E(); match(PLUS); T(); }`

Solution

(B). The first statement of `E()` is a call to `E()` with the lookahead unchanged; nothing is ever consumed, the recursion never bottoms out, and the call stack overflows. This is left recursion executed literally, the reason $E \to E + T$ must be rewritten as $E \to TE'$ before recursive descent. (A) is wrong: the function never returns, so it accepts nothing. (C) is wrong: `match(PLUS)` is never reached. (D) is wrong: no error is reported; the program crashes or loops.

Where it is in the textbooks

4.1, 4.5: `05_Parsers` §3.3 (the recursive-descent parser and the two questions each procedure asks), §3.4 (a procedure that calls itself before reading anything). 4.2: §1.2 (parse trees). 4.3: §6.4 (the stack algorithm) and §6.5 (the table *is* the recursive-descent parser). 4.4: §6.2 (empty cells mean syntax error); the comparison "which component reports first" is a consequence, not spelled out as such.

Problem 5: LL(1) table and parsing actions (13 pts)

Problem: 5(a) (8 pts): FIRST, FOLLOW, table, and whether G needs modifying

Solution

Productions numbered: 1 $S \to PQR$; 2 $P \to (S)$; 3 $P \to xSy$; 4 $P \to z$; 5 $Q \to xQ$; 6 $Q \to \varepsilon$; 7 $R \to wR$; 8 $R \to \varepsilon$. Nullable: Q, R.

	FIRST	FOLLOW
S	$\{(, x, z\}$	$\{\$,), y\}$
P	$\{(, x, z\}$	$\{x, w, \$,), y\}$
Q	$\{x, \varepsilon\}$	$\{w, \$,), y\}$
R	$\{w, \varepsilon\}$	$\{\$,), y\}$

FOLLOW(S): \$ (start),) from rule 2, y from rule 3. FOLLOW(P) = FIRST(QR) $\setminus \{\varepsilon\} \cup$ FOLLOW(S) because Q and R are both nullable. FOLLOW(Q) = FIRST(R) $\setminus \{\varepsilon\} \cup$ FOLLOW(S); FOLLOW(R) = FOLLOW(S).

	(	)	x	y	z	w	\$
S	1		1		1		
P	2		3		4		
Q		6	5	6		6	6
R		8		8		7	8

No modification is needed. There is no left recursion (S's only rule starts with P, whose rules all start with a terminal); the three alternatives of P start with distinct terminals, so no left factoring; for the nullable nonterminals, $\text{FIRST}(xQ) = \{x\}$ is disjoint from $\text{FOLLOW}(Q)$ and $\text{FIRST}(wR) = \{w\}$ from $\text{FOLLOW}(R)$. Every cell holds at most one production: G is LL(1) as given. (The x in $\text{FOLLOW}(P)$ is harmless because P has no ε-production.)

Problem: 5(b) (5 pts): parsing `( z w ) x w`

Solution

	Stack	Remaining input	Action
1	$\$\,S$	`( z w ) x w $`	$M[S, (] = 1$: push $R\,Q\,P$
2	$\$\,R\,Q\,P$	`( z w ) x w $`	$M[P, (] = 2$: push $)\,S\,($
3	$\$\,R\,Q\,)\,S\,($	`( z w ) x w $`	match (
4	$\$\,R\,Q\,)\,S$	`z w ) x w $`	$M[S, z] = 1$
5	$\$\,R\,Q\,)\,R\,Q\,P$	`z w ) x w $`	$M[P, z] = 4$
6	$\$\,R\,Q\,)\,R\,Q\,z$	`z w ) x w $`	match z
7	$\$\,R\,Q\,)\,R\,Q$	`w ) x w $`	$M[Q, w] = 6$: pop Q
8	$\$\,R\,Q\,)\,R$	`w ) x w $`	$M[R, w] = 7$: push $R\,w$
9	$\$\,R\,Q\,)\,R\,w$	`w ) x w $`	match w
10	$\$\,R\,Q\,)\,R$	`) x w $`	$M[R,)] = 8$: pop R
11	$\$\,R\,Q\,)$	`) x w $`	match)
12	$\$\,R\,Q$	`x w $`	$M[Q, x] = 5$: push $Q\,x$
13	$\$\,R\,Q\,x$	`x w $`	match x
14	$\$\,R\,Q$	`w $`	$M[Q, w] = 6$: pop Q
15	$\$\,R$	`w $`	$M[R, w] = 7$: push $R\,w$
16	$\$\,R\,w$	`w $`	match w
17	$\$\,R$	`$`	$M[R, \$] = 8$: pop R
18	$\$$	`$`	**accept**

Where it is in the textbooks

Problem 5: `05_Parsers` §4–6 and §8, and the `First_Follow_Advanced_Practice_Sheet` in `problem-sets/` (same style: nullable chains, FOLLOW through two nullable symbols).

Coverage summary

Summary: Does the textbook cover the paper?

Fully for 91 of the 100 points, and the remaining 9 are one short addition away. Every method the paper grades (derivations, ambiguity and associativity, left-recursion removal, FIRST/FOLLOW with nullable chains, the LL(1) table and the stack algorithm, recursive descent and its failure on left recursion, longest match and rule order, combined scanner DFAs, subset-construction blow-up, non-regularity arguments, nested comments) is in `05_Parsers` or `02-04_Lexical_Analysis_Complete`. Two items are not spelled out: **ε-production elimination** of a grammar

(1.6, 4 pts: the book defines nullable symbols but never does the transformation), and the **closed-form regex/DFA for `/* */` comments** (2.3(a,b), 5 pts: the book uses lexer states instead; the DFA and regex are a two-paragraph addition). The paper argues non-regularity with the pumping lemma in mind; the book's tool is distinguishable strings, which is accepted as a proof and is what the solution to 1.2 uses.